

Московский Авиационный Институт (Национальный
Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная
математика” Кафедра №806 “Вычислительная
математика и программирование”

Лабораторная работа №4 по курсу

«Операционные системы»

Группа: М8О-211Б-23

Студент: Сергеева А. А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 09.01.25

Постановка задач

Цель работы:

Приобретение практических навыков в:

- 1) Создании аллокаторов памяти и их анализу;
- 2) Создании динамических библиотек и программ, использующие динамические библиотеки.

Задание:

Исследовать два аллокатора памяти: необходимо реализовать два алгоритма аллокации памяти и сравнить их по следующим характеристикам:

- Фактор использования
- Скорость выделения блоков
- Скорость освобождения блоков
- Простота использования аллокатора

Требуется создать две динамические библиотеки, реализующие два аллокатора, соответственно. Библиотеки загружаются в память с помощью интерфейса ОС (`dlopen / LoadLibrary`) для работы с динамическими библиотеками. Выбор библиотеки, реализующей аллокатор, осуществляется чтением первого аргумента при запуске программы (`argv[1]`). Этот аргумент должен содержать путь до динамической библиотеки (относительный или абсолютный). Если аргумент не передан или по переданному пути библиотеки не оказалось, то указатели на функции, реализующие API аллокатора ниже, должны быть присвоены функциям, которые оборачивают системный аллокатор ОС (`mmap / VirtualAlloc`) в этот API. Эти аварийные оберточные функции должны быть реализованы внутри программы, которая загружает динамические библиотеки (см. пример на [GitHub Gist](#)). Каждый аллокатор памяти должен иметь функции аналогичные стандартным функциям `malloc` и `free` (`realloc`, опционально). Перед работой каждый аллокатор инициализируется свободными страницами памяти, выделенными

стандартными средствами ядра (mmap / VirtualAlloc). Необходимо самостоятельно разработать стратегию тестирования для определения ключевых характеристик аллокаторов памяти. При тестировании нужно свести к минимуму потери точности из-за накладных расходов при измерении ключевых характеристик, описанных выше.

Каждый аллокатор должен обладать следующим интерфейсом (могут быть отличия в зависимости от особенностей алгоритма):

Allocator* allocator_create(void *const memory, const size_t size)

(инициализация аллокатора на памяти memory размера size);

void allocator_destroy(Allocator *const allocator) (деинициализация структуры аллокатора);

void* allocator_alloc(Allocator *const allocator, const size_t size)

(выделение памяти аллокатором памяти размера size);

void allocator_free(Allocator *const allocator, void *const memory)

(возвращает выделенную память аллокатору);

В отчете необходимо отобразить следующее:

- Подробное описание каждого из исследуемых алгоритмов
- Процесс тестирования
- Обоснование подхода тестирования
- Результаты тестирования

Вариант 9.

Списки свободных блоков (наиболее подходящее) и алгоритм двойников.

Общий метод и алгоритм решения

Использованные системные вызовы:

- void * mmap(void *start, size_t length, int prot, int flags, int fd, off_t offset)
 - отражает length байтов, начиная со смещения offset файла (или другого объекта), определенного файловым дескриптором fd, в память, начиная с адреса start. Последний параметр (адрес) необязателен, и обычно бывает

равен 0. Настоящее местоположение отраженных данных возвращается самой функцией `mmap`. Аргумент `prot` описывает желаемый режим защиты памяти. Параметр `flags` задает тип отражаемого объекта, опции отражения и указывает, принадлежат ли отраженные данные только этому процессу или их могут читать другие.

- `int munmap(void *start, size_t length)` – освобождает ранее выделенные страницы памяти, начиная с адреса `start` и размером `length` байт, с округлением вверх до размера страницы памяти.
- `ssize_t write (int fd, const void * buf, size_t n)` – записывает `n` бит из указанного буфера в файл, соответствующий файловому дескриптору, возвращает количество записанных байт в случае успеха, иначе -1.
- `void *dlopen(const char *filename, int flag);` – Открывает динамическую библиотеку.
- `void *dlsym(void *handle, const char *symbol);` – Извлекает адрес функции или переменной `symbol` из открытой библиотеки `handle`.
- `int dlclose(void *handle);` – Закрывает динамическую библиотеку `handle`.

Аллокатор памяти на основе списка свободных блоков

Этот аллокатор использует структуру данных например, связанный список, для управления блоками памяти. Когда требуется выделить память, аллокатор ищет подходящий свободный блок в списке. После использования блоки возвращаются обратно в список.

Основные компоненты и принципы работы:

- Список свободных блоков — структура данных (обычно связанный список), хранящая указатели на свободные участки памяти.
- Блоки памяти — участки памяти, которые аллокатор выделяет или освобождает. Каждый блок может содержать информацию о размере блока и указатель на следующий свободный блок.
- Функции выделения и освобождения памяти:
 1. Выделение: поиск подходящего свободного блока в списке.
 2. Освобождение: возвращение блока в список свободных.

Алгоритм двойников

Аллокатор памяти основанный на алгоритме двойников – способ динамического распределения памяти, который помогает эффективно управлять памятью, минимизируя фрагментацию и улучшая производительность при выделении и освобождении блоков памяти.

Основные компоненты и принципы работы:

Метод использует массив или список для хранения блоков памяти по их размерам (показатели порядка или степени двойки). Каждый блок в списке представляет собой указатель на свободный блок определенного размера, и в случае выделения или освобождения памяти эти списки обновляются.

Процесс тестирования

Для тестирования использовались следующие сценарии:

1. Массовое выделение и освобождение памяти: Выделение памяти разного размера с последующим освобождением.
2. Проверка объединения блоков: Выделение нескольких блоков и освобождение их в произвольном порядке для проверки корректности объединения.
3. Измерение производительности: Сравнение времени выполнения операций выделения и освобождения памяти.
4. Измерение фрагментации: Оценка степени использования памяти.

Обоснование подхода тестирования

Тесты разработаны для проверки следующих характеристик:

1. Эффективность выделения памяти: Важно для приложений, интенсивно использующих динамическую память.
2. Корректность работы: Объединение блоков и освобождение должны работать без ошибок.
3. Производительность: Аллокатор должен минимизировать накладные расходы.
4. Фрагментация: Важно для долгосрочной работы без истощения памяти.

Результаты тестирования

Метод свободных блоков:

Производительность: Быстрое выделение памяти, но медленное объединение блоков при освобождении.

Фрагментация: Минимальная.

Память: Эффективное использование памяти для запросов любого размера.

Метод двойников

Производительность: Высокая скорость выделения и освобождения памяти.

Фрагментация: Заметная внутренняя фрагментация из-за округления размеров запросов.

Память: Эффективен для запросов, кратных степени двойки

Код программы

buddy.c - Алгоритм двойников

```
#include "library.h"
```

```
typedef struct Allocator
```

```
{  
    void *memory;  
    size_t size;  
    uint8_t *bitmap;  
    size_t block_size;  
} Allocator;
```

```
EXPORT Allocator *allocator_create(void *const memory, const size_t size)
```

```
{  
    if (!memory || size == 0)  
    {  
        return NULL;  
    }  
}
```

```
Allocator *allocator = (Allocator *)memory;
```

```
allocator->memory = (void *)((uint8_t *)memory + sizeof(Allocator));
```

```
allocator->size = size - sizeof(Allocator);
```

```
allocator->block_size = 32;
```

```
allocator->bitmap = (uint8_t *)allocator->memory;
```

```
size_t bitmap_size = allocator->size / allocator->block_size / 8;
```

```
memset(allocator->bitmap, 0, bitmap_size);
```

```
allocator->memory = (uint8_t *)allocator->bitmap + bitmap_size;
```

```

    return allocator;
}

EXPORT void allocator_destroy(Allocator *const allocator)
{
    if (allocator)
    {
        memset(allocator, 0, allocator->size);
    }
}

EXPORT void *allocator_alloc(Allocator *const allocator, const size_t size)
{
    if (!allocator || size == 0 || size > allocator->size)
    {
        return NULL;
    }

    size_t blocks_needed = (size + allocator->block_size - 1) /
allocator->block_size;

    size_t total_blocks = allocator->size / allocator->block_size;
    uint8_t *bitmap = allocator->bitmap;

    size_t free_blocks = 0;
    for (size_t i = 0; i < total_blocks; ++i)
    {
        if (!(bitmap[i / 8] & (1 << (i % 8))))
        {
            ++free_blocks;
            if (free_blocks == blocks_needed)

```



```

    {
        size_t start_block = i - blocks_needed + 1;
        for (size_t j = start_block; j <= i; ++j)
        {
            bitmap[j / 8] |= (1 << (j % 8));
        }

        return (uint8_t *)allocator->memory + start_block *
allocator->block_size;
    }
}

else
{
    free_blocks = 0;
}
}

return NULL;
}

EXPORT void allocator_free(Allocator *const allocator, void *const memory)
{
    if (!allocator || !memory)
    {
        return;
    }

    size_t offset = (uint8_t *)memory - (uint8_t *)allocator->memory;
    if (offset % allocator->block_size != 0)
    {
        return;
    }

    size_t block_index = offset / allocator->block_size;

```

```
allocator->bitmap[block_index / 8] &= ~(1 << (block_index % 8));  
}
```

freeblocks.c - Списки свободных блоков

```
#include "library.h"
```

```
#define MIN_BLOCK_SIZE 32
```

```
typedef struct Block
```

```
{  
    size_t size;  
    struct Block *next;  
    bool is_free;  
} Block;
```

```
typedef struct Allocator
```

```
{  
    Block *free_list;  
    void *memory_start;  
    size_t total_size;  
} Allocator;
```

```
EXPORT Allocator *allocator_create(void *memory, size_t size)
```

```
{  
    if (!memory || size < sizeof(Allocator))  
    {  
        return NULL;  
    }  
}
```

```

    Allocator *allocator = (Allocator *)memory;
    allocator->memory_start = (char *)memory + sizeof(Allocator);
    allocator->total_size = size - sizeof(Allocator);
    allocator->free_list = (Block *)allocator->memory_start;

    allocator->free_list->size = allocator->total_size - sizeof(Block);
    allocator->free_list->next = NULL;
    allocator->free_list->is_free = true;

    return allocator;
}

EXPORT void allocator_destroy(Allocator *const allocator)
{
    if (allocator)
    {
        memset(allocator, 0, allocator->total_size);
    }
}

EXPORT void *allocator_alloc(Allocator *allocator, size_t size)
{
    if (!allocator || size == 0)
    {
        return NULL;
    }

    size = (size + MIN_BLOCK_SIZE - 1) / MIN_BLOCK_SIZE *
    MIN_BLOCK_SIZE;

    Block *best = NULL;

```

```
Block *prev_best = NULL;
Block *current = allocator->free_list;
Block *prev = NULL;
```

```
while (current)
{
    if (current->is_free && current->size >= size)
    {
        if (best == NULL || current->size < best->size)
        {
            best = current;
            prev_best = prev;
        }
    }
    prev = current;
    current = current->next;
}
```

```
if (best)
{
    size_t remain_size = best->size - size;
    if (remain_size >= sizeof(Block) + MIN_BLOCK_SIZE)
    {
        Block *new_block = (Block *)((char *)best + sizeof(Block) + size);
        new_block->size = remain_size - sizeof(Block);
        new_block->is_free = true;
        new_block->next = best->next;

        best->next = new_block;
        best->size = size;
    }
}
```

```

    best->is_free = false;
    if (prev_best == NULL)
    {
        allocator->free_list = best->next;
    }
    else
    {
        prev_best->next = best->next;
    }

    return (void *)((char *)best + sizeof(Block));
}

return NULL;
}

EXPORT void allocator_free(Allocator *allocator, void *ptr_to_memory)
{
    if (!allocator || !ptr_to_memory)
    {
        return;
    }

    Block *head = (Block *)((char *)ptr_to_memory - sizeof(Block));
    if (!head)
        return;

    head->next = allocator->free_list;
    head->is_free = true;
    allocator->free_list = head;

```

```

Block *current = allocator->free_list;
while (current && current->next)
{
    if (((char *)current + sizeof(Block) + current->size) == (char *)current->next)
    {
        current->size += current->next->size + sizeof(Block);
        current->next = current->next->next;
    }
    else
    {
        current = current->next;
    }
}
}

```

main.c

```
#include "library.h"
```

```
#define MEMORY_POOL_SIZE 1024
```

```
static Allocator *allocator_create_stub(void *const memory, const size_t size)
```

```

{
    const char msg[] = "allocator_create: Function not found, using mmap\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    void *mapped_memory = mmap(memory, size, PROT_READ | PROT_WRITE,
MAP_PRIVATE | MAP_ANONYMOUS | MAP_FIXED, -1, 0);
    if (mapped_memory == MAP_FAILED)
    {

```

```

    const char err_msg[] = "allocator_create: mmap failed\n";
    write(STDERR_FILENO, err_msg, sizeof(err_msg) - 1);
    return NULL;
}

return (Allocator *)mapped_memory;
}

```

```

static void allocator_destroy_stub(Allocator *const allocator)
{
    const char msg[] = "allocator_destroy: Function not found, using munmap\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    if (allocator)
    {
        if (munmap(allocator, MEMORY_POOL_SIZE) == -1)
        {
            const char err_msg[] = "allocator_destroy: munmap failed\n";
            write(STDERR_FILENO, err_msg, sizeof(err_msg) - 1);
        }
    }
}

```

```

static void *allocator_alloc_stub(Allocator *const allocator, const size_t size)
{
    const char msg[] = "allocator_alloc: Function not found, using mmap\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);

```

```

    void *mapped_memory = mmap(NULL, size, PROT_READ | PROT_WRITE,
MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (mapped_memory == MAP_FAILED)
    {

```

```

    const char err_msg[] = "allocator_alloc: mmap failed\n";
    write(STDERR_FILENO, err_msg, sizeof(err_msg) - 1);
    return NULL;
}

return mapped_memory;
}

static void allocator_free_stub(Allocator *const allocator, void *const memory)
{
    const char msg[] = "allocator_free: Function not found, using munmap\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    if (memory && munmap(memory, sizeof(memory)) == -1)
    {
        const char err_msg[] = "allocator_free: munmap failed\n";
        write(STDERR_FILENO, err_msg, sizeof(err_msg) - 1);
    }
}

static allocator_create_f *allocator_create;
static allocator_destroy_f *allocator_destroy;
static allocator_alloc_f *allocator_alloc;
static allocator_free_f *allocator_free;

int main(int argc, char **argv)
{
    if (argc < 2)
    {
        const char msg[] = "Usage: ./Main <library_path>\n";
        write(STDERR_FILENO, msg, sizeof(msg));
    }
}

```



```

    return EXIT_FAILURE;
}

void *library = dlopen(argv[1], RTLD_LOCAL | RTLD_NOW);
argc++;
if (argc > 2 && library)
{
    if (!library)
    {
        const char msg[] = "Failed to load library\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        return EXIT_FAILURE;
    }

    allocator_create = dlsym(library, "allocator_create");
    allocator_destroy = dlsym(library, "allocator_destroy");
    allocator_alloc = dlsym(library, "allocator_alloc");
    allocator_free = dlsym(library, "allocator_free");

    if (!allocator_create)
    {
        allocator_create = allocator_create_stub;
    }
    if (!allocator_destroy)
    {
        allocator_destroy = allocator_destroy_stub;
    }
    if (!allocator_alloc)
    {
        allocator_alloc = allocator_alloc_stub;
    }
}

```

```

    if (!allocator_free)
    {
        allocator_free = allocator_free_stub;
    }
}
else
{
    const char msg[] = "error: failed to open custom library\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    return EXIT_FAILURE;
}

size_t size = MEMORY_POOL_SIZE;
void *addr = mmap(NULL, size, PROT_READ | PROT_WRITE,
MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (addr == MAP_FAILED)
{
    dlclose(library);
    char message[] = "mmap failed\n";
    write(STDERR_FILENO, message, sizeof(message) - 1);
    return EXIT_FAILURE;
}

Allocator *allocator = allocator_create(addr, MEMORY_POOL_SIZE);

if (!allocator)
{
    const char msg[] = "Failed to initialize allocator\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    munmap(addr, size);
    dlclose(library);
}

```

```
    return EXIT_FAILURE;
}

int *int_block = (int *)allocator_alloc(allocator, sizeof(int));
if (int_block)
{
    *int_block = 42;
    const char msg[] = "Allocated int_block with value 42\n";
    write(STDOUT_FILENO, msg, sizeof(msg));
}
else
{
    const char msg[] = "Failed to allocate memory for int_block\n";
    write(STDERR_FILENO, msg, sizeof(msg));
}

float *float_block = (float *)allocator_alloc(allocator, sizeof(float));
if (float_block)
{
    *float_block = 3.14f;
    const char msg[] = "Allocated float_block with value 3.14\n";
    write(STDOUT_FILENO, msg, sizeof(msg));
}
else
{
    const char msg[] = "Failed to allocate memory for float_block\n";
    write(STDERR_FILENO, msg, sizeof(msg));
}

if (int_block)
```

```
{
    allocator_free(allocator, int_block);
    const char msg[] = "Freed int_block\n";
    write(STDOUT_FILENO, msg, sizeof(msg));
}

if (float_block)
{
    allocator_free(allocator, float_block);
    const char msg[] = "Freed float_block\n";
    write(STDOUT_FILENO, msg, sizeof(msg));
}

allocator_destroy(allocator);
const char msg[] = "Allocator destroyed\n";
write(STDOUT_FILENO, msg, sizeof(msg));

if (library)
{
    dlclose(library);
}

munmap(addr, size);

return EXIT_SUCCESS;
}
```

Протокол работы программы

```
ali_@LAPTOP-TG8SK2OI:~/OS_2/lab_4/src$ strace ./main ./buddy.so
execve("./main", ["/main", "/buddy.so"], 0x7fff89a50fb8 /* 29 vars */) = 0
brk(NULL)                               = 0x562a5a407000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffc37a91f0) = -1 EINVAL (Invalid
argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f5400f57000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=24583, ...},
AT_EMPTY_PATH) = 0
mmap(NULL, 24583, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f5400f50000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",
O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0\0"..., 832)
= 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0"..., 48,
848) = 48
pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0I\17\357\204\3$\f\221\2039x\324\224\323\236S"..
., 68, 896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...},
AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3,
0) = 0x7f5400d27000
```

```

mprotect(0x7f5400d4f000, 2023424, PROT_NONE) = 0
mmap(0x7f5400d4f000, 1658880, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) =
0x7f5400d4f000
mmap(0x7f5400ee4000, 360448, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) =
0x7f5400ee4000
mmap(0x7f5400f3d000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) =
0x7f5400f3d000
mmap(0x7f5400f43000, 52816, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f5400f43000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f5400d24000
arch_prctl(ARCH_SET_FS, 0x7f5400d24740) = 0
set_tid_address(0x7f5400d24a10) = 7813
set_robust_list(0x7f5400d24a20, 24) = 0
rseq(0x7f5400d250e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f5400f3d000, 16384, PROT_READ) = 0
mprotect(0x562a586ae000, 4096, PROT_READ) = 0
mprotect(0x7f5400f91000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f5400f50000, 24583) = 0
getrandom("\x9c\xea\x68\x6b\x84\x21\xec\x06", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x562a5a407000
brk(0x562a5a428000) = 0x562a5a428000
openat(AT_FDCWD, "./buddy.so", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832)
= 832

```

```

newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=15632, ...},
AT_EMPTY_PATH) = 0
getcwd("/home/ali_/OS_2/lab_4/src", 128) = 26
mmap(NULL, 16432, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE,
3, 0) = 0x7f5400f52000
mmap(0x7f5400f53000, 4096, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1000) =
0x7f5400f53000
mmap(0x7f5400f54000, 4096, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) =
0x7f5400f54000
mmap(0x7f5400f55000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) =
0x7f5400f55000
close(3) = 0
mprotect(0x7f5400f55000, 4096, PROT_READ) = 0
mmap(NULL, 1024, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f5400f90000
write(1, "Allocated int_block with value 4"... , 35Allocated int_block with value 42
) = 35
write(1, "Allocated float_block with value"... , 39Allocated float_block with value
3.14
) = 39
write(1, "Freed int_block\n\0", 17Freed int_block
) = 17
write(1, "Freed float_block\n\0", 19Freed float_block
) = 19
write(1, "Allocator destroyed\n\0", 21Allocator destroyed
) = 21
munmap(0x7f5400f52000, 16432) = 0
munmap(0x7f5400f90000, 1024) = 0

```

```

exit_group(0)                = ?
+++ exited with 0 +++
ali_@LAPTOP-TG8SK2OI:~/OS_2/lab_4/src$ strace ./main ./freeblocks.so
execve("./main", ["/main", "/freeblocks.so"], 0x7fff52857828 /* 29 vars */) = 0
brk(NULL)                    = 0x55752091c000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffd11c45210) = -1 EINVAL (Invalid
argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7ff8a8954000
access("/etc/ld.so.preload", R_OK)    = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=24583, ...},
AT_EMPTY_PATH) = 0
mmap(NULL, 24583, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7ff8a894d000
close(3)                        = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",
O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0\0"..., 832)
= 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48,
848) = 48
pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0I\17\357\204\3$\f221\2039x\324\224\323\236S"..
., 68, 896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...},
AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3,

```



```

0) = 0x7ff8a8724000
mprotect(0x7ff8a874c000, 2023424, PROT_NONE) = 0
mmap(0x7ff8a874c000, 1658880, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) =
0x7ff8a874c000
mmap(0x7ff8a88e1000, 360448, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) =
0x7ff8a88e1000
mmap(0x7ff8a893a000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) =
0x7ff8a893a000
mmap(0x7ff8a8940000, 52816, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7ff8a8940000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7ff8a8721000
arch_prctl(ARCH_SET_FS, 0x7ff8a8721740) = 0
set_tid_address(0x7ff8a8721a10) = 7954
set_robust_list(0x7ff8a8721a20, 24) = 0
rseq(0x7ff8a87220e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7ff8a893a000, 16384, PROT_READ) = 0
mprotect(0x55751f81b000, 4096, PROT_READ) = 0
mprotect(0x7ff8a898e000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7ff8a894d000, 24583) = 0
getrandom("\xac\xa0\x78\x10\x95\xe5\xa8\xb7", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x55752091c000
brk(0x55752093d000) = 0x55752093d000
openat(AT_FDCWD, "./freeblocks.so", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 832) =

```

832

```
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=15640, ...},
```

```
AT_EMPTY_PATH) = 0
```

```
getcwd("/home/ali_/OS_2/lab_4/src", 128) = 26
```

```
mmap(NULL, 16432, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE,  
3, 0) = 0x7ff8a894f000
```

```
mmap(0x7ff8a8950000, 4096, PROT_READ|PROT_EXEC,  
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1000) =  
0x7ff8a8950000
```

```
mmap(0x7ff8a8951000, 4096, PROT_READ,  
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) =  
0x7ff8a8951000
```

```
mmap(0x7ff8a8952000, 8192, PROT_READ|PROT_WRITE,  
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) =  
0x7ff8a8952000
```

```
close(3) = 0
```

```
mprotect(0x7ff8a8952000, 4096, PROT_READ) = 0
```

```
mmap(NULL, 1024, PROT_READ|PROT_WRITE,
```

```
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7ff8a898d000
```

```
write(1, "Allocated int_block with value 4"..., 35Allocated int_block with value 42  
) = 35
```

```
write(1, "Allocated float_block with value"..., 39Allocated float_block with value  
3.14  
) = 39
```

```
write(1, "Freed int_block\n\0", 17Freed int_block  
) = 17
```

```
write(1, "Freed float_block\n\0", 19Freed float_block  
) = 19
```

```
write(1, "Allocator destroyed\n\0", 21Allocator destroyed  
) = 21
```

```
munmap(0x7ff8a894f000, 16432) = 0
```

```

munmap(0x7ff8a898d000, 1024)      = 0
exit_group(0)                    = ?
+++ exited with 0 +++
ali_@LAPTOP-TG8SK2OI:~/OS_2/lab_4/src$ strace ./main
execve("./main", [ "./main" ], 0x7ffd299d83f0 /* 29 vars */) = 0
brk(NULL)                        = 0x559e653d3000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffd437526d0) = -1 EINVAL (Invalid
argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f10ac219000
access("/etc/ld.so.preload", R_OK)  = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=24583, ...},
AT_EMPTY_PATH) = 0
mmap(NULL, 24583, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f10ac212000
close(3)                          = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",
O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0\0"..., 832)
= 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48,
848) = 48
pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0I\17\357\204\3$\f\221\2039x\324\224\323\236S"..
., 68, 896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...},
AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784

```

```

mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3,
0) = 0x7f10abfe9000
mprotect(0x7f10ac011000, 2023424, PROT_NONE) = 0
mmap(0x7f10ac011000, 1658880, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) =
0x7f10ac011000
mmap(0x7f10ac1a6000, 360448, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) =
0x7f10ac1a6000
mmap(0x7f10ac1ff000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) =
0x7f10ac1ff000
mmap(0x7f10ac205000, 52816, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f10ac205000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f10abfe6000
arch_prctl(ARCH_SET_FS, 0x7f10abfe6740) = 0
set_tid_address(0x7f10abfe6a10) = 8032
set_robust_list(0x7f10abfe6a20, 24) = 0
rseq(0x7f10abfe70e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f10ac1ff000, 16384, PROT_READ) = 0
mprotect(0x559e63501000, 4096, PROT_READ) = 0
mprotect(0x7f10ac253000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f10ac212000, 24583) = 0
write(2, "Usage: ./Main <library_path>\n\n0", 30Usage: ./Main <library_path>
) = 30
exit_group(1) = ?
+++ exited with 1 +++

```

Вывод

В ходе выполнения данной лабораторной работы я научилась работе с динамическими библиотеками, изучила новые системные вызовы, для их использования. Я научилась подключать динамические библиотеки, обрабатывать ошибки, связанные с их подключением, и использовать их.

В ходе написания данной лабораторной работы я узнала об устройстве аллокаторов и реализовала два алгоритма аллокации памяти, работающие через один API и подключаемые через динамические библиотеки.